Bài 2.2 – Linear Regression & Loss Function

> **Giai đoạn 2 – Machine Learning cổ điển** · Thời lượng: 2 buổi · Hình thức mở đầu: *Đố vui trước khi học*

🎯 Sau bài này bạn sẽ

- Hiểu **Linear Regression (Hồi quy tuyến tính)** từ trực giác đến công thức, tự tính tay được.
- Hiểu vì sao loss hay dùng **bình phương** sai số.
- Dùng Scikit-learn cho hồi quy một biến và nhiều biến, **đọc được ý nghĩa** từng hệ số.
- Xử lý cột chữ (one-hot, `drop_first`) và đọc hệ số của chúng.
- Kiểm tra mô hình bằng **biểu đồ phần dư** và biết giới hạn của mô hình tuyến tính.

Chia buổi:

- **Buổi 1:** Mục 1 → 5 (một biến, loss, công thức, so sánh với Gradient Descent).
- **Buổi 2:** Mục 6 → 10 (nhiều biến, hệ số, cột chữ, phần dư, bài tập).

---

1. Đố vui trước khi học 🤔

Bạn có 5 căn hộ (diện tích tính bằng chục m², giá tính bằng tỷ). Ba bạn đề xuất ba đường thẳng để dự đoán giá:

Căn	x	y thật	An: ŷ = 0,4x + 0,7	Bình: ŷ = 0,5x + 0,02	Chi: ŷ = 0,6x − 0,5
1	4	2,0	2,30	2,02	1,90
2	5	2,6	2,70	2,52	2,50
3	6	2,9	3,10	3,02	3,10
4	7	3,6	3,50	3,52	3,70
5	8	4,0	3,90	4,02	4,30

Câu hỏi: Đường của ai tốt nhất? Bạn dựa vào tiêu chí gì để nói "tốt nhất"?

Tự nghĩ 2 phút trước khi đọc tiếp. Câu hỏi thứ hai mới là câu hỏi quan trọng: **phải định nghĩa "tốt" bằng một con số** thì máy mới tìm được.

Đáp án

> Mẹo Notion: chọn tiêu đề "Đáp án" → `Turn into → Toggle heading 3`.

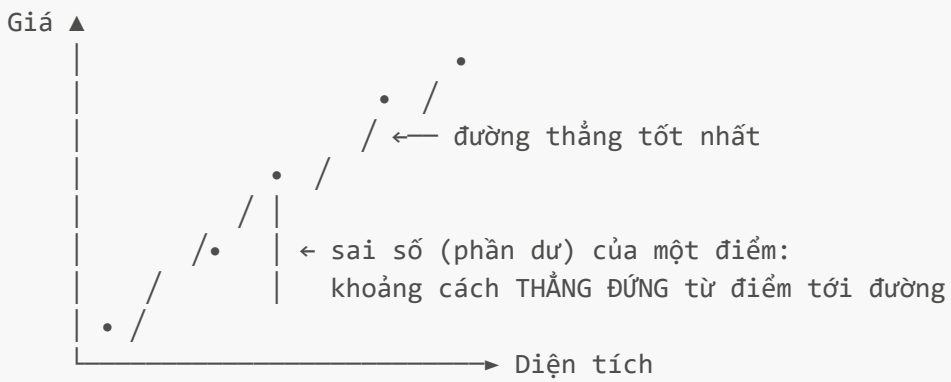
Tính tổng **bình phương sai số** $(\hat{y} - y)^2$ cho mỗi người:

- An: 0,09 + 0,01 + 0,04 + 0,01 + 0,01 = **0,16**
- Bình: 0,0004 + 0,0064 + 0,0144 + 0,0064 + 0,0004 = **0,028**
- Chi: 0,01 + 0,01 + 0,04 + 0,01 + 0,09 = **0,16**

Bình thắng. Thật ra đường của Bình là đường **tốt nhất có thể** theo tiêu chí này. Bạn đã tìm ra nó bằng Gradient Descent ở bài 1.9 (w = 0,5; b = 0,02). Bài này cho bạn cách tính nó trực tiếp.

---

2. Trực giác: kẻ đường thẳng "xuyên qua" đám mây điểm



Linear Regression:

- **Định nghĩa:** Tìm đường thẳng (hoặc mặt phẳng, khi có nhiều đặc trưng) sao cho **tổng bình phương khoảng cách thẳng đứng** từ các điểm tới đường là nhỏ nhất.
- **Ví dụ đời thực:** Môi giới ước giá: "khu này mỗi m^2 khoảng 50 triệu, cộng thêm vài trăm triệu tiền vị trí".

---

3. Mô hình và hàm loss

3.1. Mô hình

$$\hat{y} = w \cdot x + b$$

Ký hiệu	Ý nghĩa	Ví dụ
x	Đặc trưng	Diện tích (chục m^2)
w	**Hệ số góc (slope / weight)**: x tăng 1 thì ŷ tăng w	0,5 tỷ mỗi chục m^2
b	**Hệ số chặn (intercept / bias)**: giá trị ŷ khi x = 0	0,02 tỷ
ŷ	Giá dự đoán	

3.2. Loss Function (Hàm mất mát) – MSE

- **Định nghĩa:** Hàm biến "mô hình sai bao nhiêu" thành **một con số**. Học = tìm tham số làm con số này nhỏ nhất.
- **Ví dụ đời thực:** Điểm phạt trong thi bắn cung: bắn càng xa hồng tâm, phạt càng nặng.

$$MSE = \frac{1}{n} \sum (y_i - \hat{y}_i)^2$$

```
n  i=1
```

Vì sao bình phương mà không lấy trị tuyệt đối?

Lý do	Giải thích
Phạt nặng lỗi lớn	Sai 2 tỷ bị phạt 4, sai 4 tỷ bị phạt 16 (gấp 4 lần, không phải 2)
Đạo hàm đẹp	$(u^2)' = 2u$, mượt ở mọi nơi → Gradient Descent chạy tốt (bài 1.9). Trị tuyệt đối bị "gãy" ở 0
Có công thức nghiệm trực tiếp	Mục 4

Cái giá phải trả: **rất nhạy với outlier** (một điểm lệch xa bị bình phương thành con số khổng lồ, kéo cả đường thẳng về phía nó).

---

4. Công thức tính trực tiếp (một biến)

Với MSE, ta không cần Gradient Descent: có công thức cho nghiệm tốt nhất, gọi là **OLS – Ordinary Least Squares (Bình phương tối thiểu)**.

$$w = \frac{\Sigma (x_i - \bar{x})(y_i - \bar{y})}{\Sigma (x_i - \bar{x})^2} \qquad b = \bar{y} - w \cdot \bar{x}$$

Tử số giống hệt tử số của hệ số tương quan (bài 1.7). Thật vậy, w và r có liên hệ chặt chẽ: w = r × (độ lệch chuẩn của y / độ lệch chuẩn của x).

Tính tay với 5 căn hộ: $\bar{x} = 6$, $\bar{y} = 3{,}02$.

x	y	x − 6	y − 3,02	tích	$(x-6)^2$
4	2,0	−2	−1,02	2,04	4
5	2,6	−1	−0,42	0,42	1
6	2,9	0	−0,12	0	0
7	3,6	1	0,58	0,58	1
8	4,0	2	0,98	1,96	4
			Tổng	**5,0**	**10**

- w = 5,0 / 10 = **0,5**
- b = 3,02 − 0,5 × 6 = **0,02**

---

5. Code: một biến

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression

x = np.array([4, 5, 6, 7, 8], dtype=float)          # diện tích (chục m²)
y = np.array([2.0, 2.6, 2.9, 3.6, 4.0])             # giá (tỷ)

mh = LinearRegression().fit(x.reshape(-1, 1), y)    # sklearn cần X 2 chiều (bài 1.3)
print(mh.coef_, round(mh.intercept_, 4))            # coef_: các w; intercept_: b
print(mh.predict([[6.5]]))                          # dự đoán căn 65 m²

# Tự tính bằng công thức OLS để đối chiếu
w = ((x - x.mean()) * (y - y.mean())).sum() / ((x - x.mean()) ** 2).sum()
b = y.mean() - w * x.mean()
print(w, round(b, 4))
```

Kết quả mẫu:

```
[0.5] 0.02
[3.27]
0.5 0.02
```

Ba cách cho cùng một đáp án: tự viết Gradient Descent (bài 1.9), công thức OLS, và Scikit-learn.

Cách	Ưu	Nhược
Công thức OLS (sklearn `LinearRegression` dùng cách tương tự)	Chính xác, không cần chọn learning rate	Chậm khi có hàng triệu dòng × hàng nghìn cột
Gradient Descent	Chạy được với dữ liệu khổng lồ, dùng được cho mọi mô hình (mạng nơ-ron)	Cần chọn learning rate, cần chuẩn hóa

---

6. Nhiều biến: Multiple Linear Regression

6.1. Mô hình

$$\hat{y} = w_1x_1 + w_2x_2 + \ldots + w_nx_n + b = w \cdot x + b$$

Đây chính là tích vô hướng ở bài 1.8. Với 2 đặc trưng, mô hình là một **mặt phẳng** trong không gian 3 chiều. Với nhiều hơn, không hình dung được nữa, nhưng toán y hệt.

6.2. Bắt đầu với một biến trên dữ liệu thật

Dùng ô `nap_du_lieu_nha()` ở bài 2.1 (mục 5.1).

```
from sklearn.model_selection import train_test_split

df = nap_du_lieu_nha()
y = df["gia_nha"]                                    # nghìn USD

X1 = df[["thu_nhap"]]
Xtr, Xte, ytr, yte = train_test_split(X1, y, test_size=0.2, random_state=42)
m1 = LinearRegression().fit(Xtr, ytr)
print(m1.coef_.round(2), round(m1.intercept_, 2), round(m1.score(Xte, yte), 3))
```

Kết quả mẫu:

```
[40.24] 44.43 0.401
```

Đọc: giá ≈ 40,24 × thu_nhap + 44,43. Thu nhập tăng 1 đơn vị (10.000 USD/năm) → giá nhà tăng khoảng **40.000 USD**. R^2 = 0,40.

6.3. Sáu đặc trưng

```
cot = ["thu_nhap", "km_tp_lon", "phong_moi_ho", "ti_le_phong_ngu", "nguoi_moi_ho",
"tuoi_nha"]
Xtr, Xte, ytr, yte = train_test_split(df[cot], y, test_size=0.2, random_state=42)
m2 = LinearRegression().fit(Xtr, ytr)
print(pd.Series(m2.coef_, index=cot).round(3))       # ghép hệ số với tên cột cho
dễ đọc
print(round(m2.intercept_, 2), round(m2.score(Xte, yte), 3))
```

Kết quả mẫu:

```
thu_nhap            46.920
km_tp_lon           -0.257
phong_moi_ho         0.762
ti_le_phong_ngu    381.060
nguoi_moi_ho        -0.497
tuoi_nha             0.907
dtype: float64
-66.75 0.537
```

R^2 tăng từ 0,40 lên **0,537**.

6.4. Đọc hệ số: "khi mọi thứ khác giữ nguyên"

Hệ số	Đọc là
thu_nhap = 46,9	Hai khu **giống hệt nhau về mọi mặt khác**, khu có thu nhập cao hơn 1 đơn vị thì giá cao hơn khoảng 46.900 USD
km_tp_lon = −0,257	Xa thành phố lớn thêm 1 km → giá giảm khoảng 257 USD. Xa thêm 100 km → giảm khoảng 25.700 USD
tuoi_nha = 0,907	Nhà cũ hơn 1 năm → giá cao hơn ~900 USD (khi đã tính tới thu nhập và vị trí)
ti_le_phong_ngu = 381	⚠️ Xem bên dưới

Cụm từ quan trọng nhất: "khi mọi thứ khác giữ nguyên". Để ý hệ số của thu_nhap tăng từ 40,2 (một biến) lên 46,9 (sáu biến): khi đã tính riêng tác động của vị trí, tác động "thuần" của thu nhập thay đổi.

Bẫy: `ti_le_phong_ngu` có hệ số 381, trông như quan trọng nhất. **Sai!** Cột này chỉ dao động 0,1 – 0,3 nên tăng "1 đơn vị" (từ 0,2 lên 1,2) là vô nghĩa. Còn `km_tp_lon` dao động tới hàng trăm. **Không so sánh độ lớn hệ số khi các cột khác đơn vị.**

6.5. Muốn so sánh tầm quan trọng: chuẩn hóa trước

```
from sklearn.preprocessing import StandardScaler

sc = StandardScaler().fit(Xtr)                          # fit CHỈ trên train (bài
1.11)
m3 = LinearRegression().fit(sc.transform(Xtr), ytr)
print(pd.Series(m3.coef_, index=cot).round(2).sort_values(key=abs,
ascending=False))
print(round(m3.score(sc.transform(Xte), yte), 3))
```

Kết quả mẫu:

```
thu_nhap           74.33
km_tp_lon         -21.79
ti_le_phong_ngu    21.48
tuoi_nha           11.40
nguoi_moi_ho       -2.26
phong_moi_ho        1.79
dtype: float64
0.537
```

Giờ mỗi hệ số nghĩa là: "đặc trưng tăng **1 độ lệch chuẩn** thì giá thay đổi bao nhiêu nghìn USD". Có thể so sánh: **thu nhập quan trọng nhất** (74), rồi khoảng cách (−22). R^2 **không đổi** (0,537): chuẩn hóa không làm Linear Regression tốt hơn, chỉ làm hệ số dễ so sánh.

6.6. Dự đoán cho một khu mới

```
khu_moi = pd.DataFrame([{"thu_nhap": 5.0, "km_tp_lon": 20, "phong_moi_ho": 5.5,
                         "ti_le_phong_ngu": 0.2, "nguoi_moi_ho": 2.8, "tuoi_nha":
25}])
print(m2.predict(khu_moi).round(1))     # cột phải đúng tên và thứ tự như lúc fit
```

Kết quả mẫu:

```
[264.4]
```

Khu thu nhập 50.000 USD, cách thành phố 20 km → giá dự đoán khoảng **264.400 USD**.

---

7. Cột chữ trong Linear Regression

Ví dụ nhỏ: giá căn hộ theo quận và diện tích.

```
nha = pd.DataFrame({"quan": ["Q1", "Q7", "Q1", "Thủ Đức", "Q7"],
                    "dien_tich": [50, 80, 60, 90, 70],
                    "gia": [6.5, 5.6, 7.2, 4.4, 5.0]})
# drop_first=True: bỏ cột one-hot của nhóm đầu tiên (Q1) → Q1 thành "nhóm gốc"
Xd = pd.get_dummies(nha[["quan", "dien_tich"]], columns=["quan"], drop_first=True,
dtype=int)
print(Xd)
md = LinearRegression().fit(Xd, nha["gia"])
print(pd.Series(md.coef_, index=Xd.columns).round(3), round(md.intercept_, 3))
```

Kết quả mẫu:

```
   dien_tich  quan_Q7  quan_Thủ Đức
0         50        0             0
1         80        1             0
2         60        0             0
3         90        0             1
4         70        1             0
dien_tich       0.065
quan_Q7        -2.850
quan_Thủ Đức   -4.725
dtype: float64 3.275
```

Đọc hệ số one-hot: so với **nhóm gốc** (Q1, nhóm bị bỏ cột):

- Căn ở Q7 **rẻ hơn** căn Q1 cùng diện tích khoảng **2,85 tỷ**.
- Căn ở Thủ Đức rẻ hơn căn Q1 cùng diện tích khoảng **4,7 tỷ**.

Vì sao `drop_first`? Nếu giữ đủ 3 cột, ba cột luôn cộng lại bằng 1, trùng hoàn toàn với hệ số chặn b. Mô hình không biết chia "tiền nền" cho ai, hệ số trở nên không ổn định. Bỏ 1 cột (bài 1.6: k nhóm chỉ cần k−1 cột) giải quyết vấn đề này.

> ⚠️ 5 căn hộ là quá ít để tin các con số. Đây chỉ là ví dụ minh họa cách đọc hệ số.

---

8. Kiểm tra mô hình bằng phần dư

- **Residual (Phần dư):** y thật − ŷ dự đoán. Phần dư dương: mô hình đoán **thấp** hơn thật.

```
import matplotlib.pyplot as plt

du_doan = m2.predict(Xte)
phan_du = yte - du_doan
print(round(phan_du.mean(), 2), (du_doan < 0).sum())     # trung bình phần dư; số dự đoán giá ÂM

fig, ax = plt.subplots(figsize=(7, 4))
ax.scatter(du_doan, phan_du, s=3, alpha=0.3)
ax.axhline(0, color="red")                                # đường phần dư = 0
ax.set_xlabel("Giá dự đoán (nghìn USD)"); ax.set_ylabel("Phần dư")
ax.set_title("Biểu đồ phần dư")
plt.show()
```

Kết quả mẫu:

```
-0.26 1
```

Bạn sẽ thấy: đám mây điểm quanh đường 0, nhưng **không đều**:

- Có những **đường biên chéo** rõ rệt ở góc dưới bên trái và góc trên bên phải. Lý do: giá thật bị chặn trong khoảng 15–500 nghìn USD (ta đã bỏ khu trên 500), nhưng đường thẳng thì không biết giới hạn đó.
- Có **1 khu bị dự đoán giá âm**! Đường thẳng cứ kéo dài mãi, không biết giá nhà không thể âm.

Biểu đồ phần dư lý tưởng: một dải ngẫu nhiên, đều, đối xứng quanh 0, không có hình dạng. Có hình dạng (đường cong, hình phễu, đường biên) = mô hình đang **bỏ sót** một quy luật nào đó.

---

9. Ưu, nhược điểm

Ưu điểm	Nhược điểm
Rất nhanh, dễ hiểu, dễ giải thích cho sếp	Chỉ học được quan hệ **thẳng**
Hệ số có ý nghĩa rõ ràng	Rất nhạy với outlier (do bình phương)

Ưu điểm	Nhược điểm
Mốc tốt để so sánh mô hình phức tạp hơn	Hệ số không ổn định khi các đặc trưng tương quan mạnh với nhau (sẽ xử lý ở bài 2.6)
	Có thể dự đoán giá trị vô lý (giá âm)

Muốn học quan hệ cong mà vẫn dùng Linear Regression? Tạo thêm đặc trưng như x^2, log(x) (bài 2.6 và 2.16). Hoặc dùng mô hình dạng cây (bài 2.9–2.11).

---

10. Bài tập

1. **Tính tay** w và b cho dữ liệu: x = [1, 2, 3, 4], y = [3, 5, 7, 9]. Kiểm tra bằng sklearn. Dự đoán x = 10.
2. Thêm `kinh_do` và `vi_do` vào 6 đặc trưng ở mục 6.3. R^2 thay đổi thế nào?
3. Huấn luyện mô hình dự đoán **log của giá** (`np.log(ytr)`), rồi đổi ngược bằng `np.exp()` khi dự đoán. So sánh MAE với mô hình gốc (MAE tính bằng `mean_absolute_error` từ `sklearn.metrics`). Có còn dự đoán giá âm không?
4. Trong ví dụ quận ở mục 7, dự đoán giá căn **Q7, 75 m²**. Tính tay từ hệ số, rồi kiểm tra bằng `md.predict`.

Đáp án

> Mẹo Notion: chọn tiêu đề "Đáp án" → `Turn into → Toggle heading 3`.

1. $\bar{x}$ = 2,5; $\bar{y}$ = 6. Độ lệch x: −1,5; −0,5; 0,5; 1,5. Độ lệch y: −3; −1; 1; 3. Tử số = 4,5 + 0,5 + 0,5 + 4,5 = 10. Mẫu số = 2,25 + 0,25 + 0,25 + 2,25 = 5. **w = 2**, **b = 6 − 2 × 2,5 = 1**. Dự đoán x = 10: 2 × 10 + 1 = **21**. (Dữ liệu nằm hoàn hảo trên đường thẳng y = 2x + 1.)

```
# 2
cot2 = cot + ["kinh_do", "vi_do"]
Xtr2, Xte2, ytr2, yte2 = train_test_split(df[cot2], y, test_size=0.2,
random_state=42)
print(round(LinearRegression().fit(Xtr2, ytr2).score(Xte2, yte2), 3))
# R² tăng từ 0.537 lên 0.587 vì kinh độ/vĩ độ bổ sung thông tin vị trí mà
km_tp_lon chưa có
# (ví dụ: ven biển hay nội địa).

# 3
from sklearn.metrics import mean_absolute_error
m4 = LinearRegression().fit(Xtr, np.log(ytr))
du_doan_log = np.exp(m4.predict(Xte))                    # exp: đổi ngược log
print(round(mean_absolute_error(yte, du_doan), 1),       # 48.7
      round(mean_absolute_error(yte, du_doan_log), 1))   # 48.3
print((du_doan_log < 0).sum())                           # 0: exp() luôn dương
```

3. MAE giảm nhẹ (48,7 → 48,3) và **không còn giá âm**, vì e mũ bất kỳ số nào cũng dương. Mẹo log cột đích rất hay dùng khi cột đích lệch phải và luôn dương.

4. Tay: 3,275 + 0,065 × 75 + (−2,85) × 1 + (−4,725) × 0 = 3,275 + 4,875 − 2,85 = **5,3 tỷ**.

```
moi = pd.DataFrame([{"dien_tich": 75, "quan_Q7": 1, "quan_Thủ Đức": 0}])
print(md.predict(moi).round(2))     # ≈ 5.3
```

---

11. Flashcard

Mặt trước	**Mặt sau**
Linear Regression tìm gì?	Đường thẳng/mặt phẳng có tổng bình phương phần dư nhỏ nhất.
Công thức OLS cho w (một biến)?	$\Sigma(x-\bar{x})(y-\bar{y}) / \Sigma(x-\bar{x})^2$; $b = \bar{y} - w\cdot\bar{x}$
Vì sao loss dùng bình phương?	Phạt nặng lỗi lớn, đạo hàm mượt, có nghiệm trực tiếp.
Nhược điểm của bình phương?	Rất nhạy với outlier.
Đọc hệ số w của đặc trưng x?	x tăng 1 đơn vị, các đặc trưng khác giữ nguyên → ŷ tăng w.
So sánh tầm quan trọng các hệ số thế nào?	Chuẩn hóa đặc trưng trước, rồi so trị tuyệt đối hệ số.
Vì sao `drop_first=True`?	Tránh các cột one-hot trùng thông tin với hệ số chặn.
Hệ số one-hot đọc thế nào?	Chênh lệch so với nhóm gốc (nhóm bị bỏ cột).
Phần dư là gì?	y thật − ŷ dự đoán.
Biểu đồ phần dư tốt trông thế nào?	Dải ngẫu nhiên đều quanh 0, không có hình dạng.

Bài tiếp theo: 2.3 – Đánh giá hồi quy: MAE, MSE, RMSE, R^2.